Result | Size | Time | Cycles | GPU | SM Frequency | Process | Attributes
0 | 0% - gemm_wmma_kernel | 0 | 25.3, 14,118.4, 57.7 ms | 136,598 | 0 | NVIDIA GeForce RTX 4090 Laptop GPU | 1.62 GHz | [1664.00, gemm_kernel

Summary | Details | Source | Occupancy | Communities | Raw | Sessions

Compare | Note | View | Export

GPU Speed Of Light Throughput

Roofline Tensor Core

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of the unit. Depending on the hardware architecture, the throughput of individual units can be used to derive the achieved compute and memory throughput.

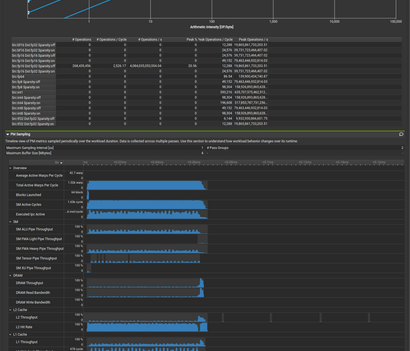
Compute (SM) Throughput [%]	25.15	Elapsed Cycles [cycle]	136,345
Memory Throughput [%]	29.15	Elapsed Cycles [cycle]	136,345
L1/TEX Cache Throughput [%]	57.72	SM Active Cycles [cycle]	102,191.52
L2 Cache Throughput [%]	19.39	Memory Throughput [Gbyte/s]	1.66
DRAM Throughput [%]	5.39	DRAM Frequency [Ghz]	1.66

Small Grid - The kernel grid is too small to fill the available resources on this device, resulting in only 0.5 full waves across all SMs. Look at Launch Statistics for more details.

Key Performance Indicators

Roofline Analysis - The ratio of peak float (FP32) to double (FP64) performance on this device is 64:1. The workload achieved close to 0% of this device's FP32 peak performance and 0% of its FP64 peak performance. See the Profiling Guide for more details on roofline analysis.

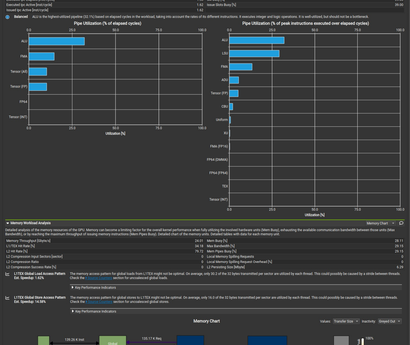
Tensor Core Operations Roofline



PM Sampling

Timeline view of PM counters sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

Maximum Sampling Interval [us]	4 Pass Groups
Maximum Buffer Size [MiBytes]	1



Compute Workload Analysis

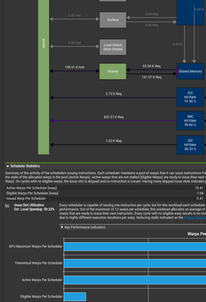
Pipe Utilization (Elapsed Cycles)

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

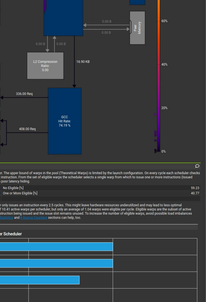
Executed Ipc Elapsed [inst/cycle]	0.06
Executed Ipc Active [inst/cycle]	0.25
Issued Ipc Active [inst/cycle]	0.25

Balanced - ALU is the highest-utilized pipeline (57.7%) based on elapsed cycles in the workload, taking into account the issue rates of its different instructions. It executes integer and logic operations. It is well-utilized, but should not be a bottleneck.

Pipe Utilization (% of elapsed cycles)



Pipe Utilization (% of peak instructions executed over elapsed cycles)



Memory Workload Analysis

Memory Chart

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed charts of memory units and the connections between them are shown in the Memory Chart.

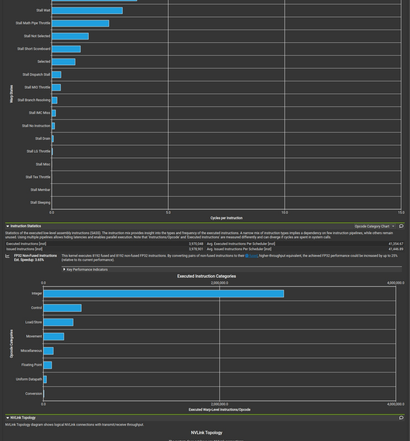
Memory Throughput [Gbyte/s]	24.01	Mem Busy [%]	29.11
L1/TEX Hit Rate [%]	75.72	Max Bandwidth [%]	57.72
L2 Hit Rate [%]	19.15	Mem Pipes Busy [%]	29.15
L2 Compression Input Sectors [sector]	0	Local Memory Spilling Requests	0
L2 Compression Ratio	0	Local Memory Spilling Request Overhead [%]	0
L2 Compression Success Rate [%]	0	Shared Memory Spilling [byte]	0

L1/TEX Global Load Access Pattern - The memory access pattern for global loads from L1/TEX might not be optimal. On average, only 16.0 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Memory Workload Analysis section for uncoalesced global loads.

L1/TEX Global Store Access Pattern - The memory access pattern for global stores to L1/TEX might not be optimal. On average, only 16.0 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Memory Workload Analysis section for uncoalesced global stores.

Key Performance Indicators

Memory Chart



Scheduler Statistics

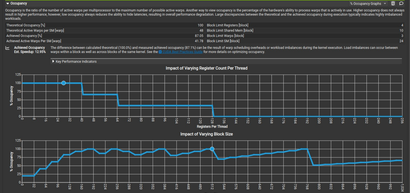
Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks how many warps in its pool are stalled (Waiting, Dispatch, Scoreboard, etc.) preventing their next instruction to be issued (Stall). Each cycle with no instruction issued (Issue) is a scheduler cycle in which no warp was eligible. Ideally, every cycle should issue one instruction, so to hide all latencies the number of eligible warps per scheduler should be sufficiently larger than one. Hiding every latency usually requires a high number of eligible warps, which is impossible if each instruction is dependent on the previous one. In order to increase the number of eligible warps, avoid possible load imbalances due to e.g. varying amounts of work per warp scheduler, or drastically reduce the latency between dependent instructions, the compiler can try to increase the number of independent instructions between dependent ones.

One or More Eligible	0.25
Issued Warp Per Scheduler	0.25
No Eligible	99.75
Active Warps Per Scheduler	4.17
Eligible Warps Per Scheduler	0.01

Issue Slot Utilization - Every scheduler is capable of issuing one instruction per cycle, but for this workload each scheduler only issues an instruction every 4.0 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 12 warps per scheduler, this workload allocates an average of 4.17 active warps per scheduler, but only an average of 0.01 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to varying amounts of work per warp scheduler. Reduce the latency between dependent instructions, for example by increasing the number of independent instructions within each warp.

Key Performance Indicators

Warps Per Scheduler



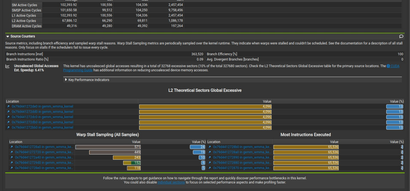
Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles that warps were in that state per issued instruction. Stalled warps are not issuing because they are waiting for a specific condition. When executing a kernel with issued Warp Cycles Per Issued Instruction 4.00 Active Threads Per Warp 32 Active Warps Per Scheduler 4.17 No Eligible 99.75 Eligible Warps Per Scheduler 0.01 Issued Warp Per Scheduler 0.25 Issue Slot Utilization 6.25% Every cycle with no issued instruction (Issue Slot Utilization 6.25%) at least one warp could not issue its next instruction. To increase the number of eligible warps, avoid possible load imbalances due to varying amounts of work per warp scheduler. Reduce the latency between dependent instructions, for example by increasing the number of independent instructions within each warp. Each cycle no eligible warp (Eligible Warps Per Scheduler 0.01) results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to varying amounts of work per warp scheduler. Reduce the latency between dependent instructions, for example by increasing the number of independent instructions within each warp. Issue Slot Utilization 6.25% Every cycle with no issued instruction (Issue Slot Utilization 6.25%) at least one warp could not issue its next instruction. To increase the number of eligible warps, avoid possible load imbalances due to varying amounts of work per warp scheduler. Reduce the latency between dependent instructions, for example by increasing the number of independent instructions within each warp. Each cycle no eligible warp (Eligible Warps Per Scheduler 0.01) results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to varying amounts of work per warp scheduler. Reduce the latency between dependent instructions, for example by increasing the number of independent instructions within each warp.

Key Performance Indicators

Warp Stall - Check the Source Counters table for the top stall locations in your source based on sampling data. The Profiling Guide provides more details on each stall reason.

Warp State (All Cycles)



Instruction Statistics

Opcode Category Chart

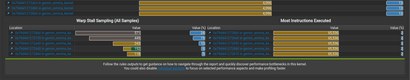
Statistics of the executed low-level assembly instructions (SASS). The Instruction mix provides insight into the type and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain idle. Combining several pipelines and utilizing a balanced mix hides instruction and execution latencies. Note that Instructions Executed and Executed Instructions are measured differently and can diverge if some work is spilled to stack.

Executed Instructions [inst]	3,570,048	Avg. Executed Instructions Per Scheduler [inst]	41,204.47
Executed Ipc Active [inst/cycle]	0.25	Executed Instructions Per Scheduler [inst]	41,066.88
Executed Ipc Elapsed [inst/cycle]	0.06	Avg. Issued Instructions Per Scheduler [inst]	41,244.84
Issue Slots Busy [%]	6.25	Issued Instructions Per Scheduler [inst]	41,111.88

FP32 Instruction Mix - The kernel executes 3570432 fused and 6192 non-fused FP32 instructions. By converting pairs of non-fused into fused instructions the SM can reach higher throughput equivalent, the achieved FP32 performance could be increased by up to 25% (relative to its current performance).

Key Performance Indicators

Executed Instruction Categories



NVLink Topology

NVLink Topology

NVLink Topology Diagram shows logical NVLink connections with transmit/receive throughput.

The system does not have any NVLink connections.

NVLink Tables

NVLink Tables

Detailed tables with properties for each NVLink.

Logical NVLink Properties

The system does not have any NVLink connections.

NUMA Affinity

NUMA Table

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

GPU ID / GPU Name	NUMA ID by CPU Affinity	GPU Affinity NUMA ID by Memory Affinity
0 - NVIDIA GeForce RTX 4090 Laptop GPU	0	0

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	64	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	38	Static Shared Memory Per Block [Kbyte/block]	0
Block Size	512	Dynamic Shared Memory Per Block [Kbyte/block]	0
# Threads [thread]	32,768	Driver Shared Memory Per Block [Kbyte/block]	1.02
Threads Per Wave	32,768	Shared Memory Configuration Size [Kbyte]	65.54
Uses Green Context	0	Stack Size	1,024
# Waves [wave]	64	# TPCs	12
Enabled TPC IDs	all		

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy can typically indicate highly imbalanced workloads.

Theoretical Occupancy [%]	100	Block Limit Registers [block]	3
Theoretical Active Warps per SM [warp]	64	Block Limit Shared Mem [block]	32
Achieved Occupancy [%]	87.01	Block Limit Warps [block]	3
Achieved Active Warps Per SM [warp]	55.69	Block Limit SM [block]	24

Achieved Occupancy - The difference between calculated theoretical (100.0%) and measured achieved occupancy (87.1%) can be the result of warp scheduling overheads or workload imbalances during the kernel execution. Load imbalances can occur between warps within a block as well as between different blocks of the same kernel. See the Profiling Guide for more details on occupancy and its performance implications.

Key Performance Indicators

Impact of Varying Register Count Per Thread

Impact of Varying Block Size

Impact of Varying Shared Memory Usage Per Block

GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of the SM, SMIF, SMSP, L1 & L2 caches, and DRAM.

Average SM Active Cycles [cycle]	102,202.52	Average L1 Active Cycles [cycle]	102,295.52
Average L2 Active Cycles [cycle]	87,992.12	Average SMSP Active Cycles [cycle]	101,630.58
Average DRAM Active Cycles [cycle]	29,696	Total SM Elapsed Cycles [cycle]	1,640,160
Total DRAM Elapsed Cycles [cycle]	1,091,584	Total L1 Elapsed Cycles [cycle]	1,639,200
Total SMSP Elapsed Cycles [cycle]	13,121,120	Total L2 Elapsed Cycles [cycle]	2,183,216

Workload Distribution

	Average	Min	Max	Unit
SM Active Cycles	102,202.52	101,630.58	2,457,888	cycle
SMSP Active Cycles	101,630.58	99,512	104,200	cycle
L1 Active Cycles	102,202.52	100,556	104,556	cycle
L2 Active Cycles	87,992.12	46,294	98,431	cycle
DRAM Active Cycles	29,696	25,283	44,296	cycle

Source

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only a limited number of entries can be listed per metric type.

Uncoalesced Global Accesses	0	Theoretical Global Sector Reductions	0
Est. Speedup [%]	0	Avg. Divergent Branches [branches]	0

Uncoalesced Global Accesses - This kernel has uncoalesced global accesses resulting in a total of 0 excessive sectors (0% of the total 0 sectors). Check the L2 Theoretical Sectors Global Excessive table for the primary source locations. The Profiling Guide has additional information on reducing uncoalesced device memory accesses.

Key Performance Indicators

L2 Theoretical Sectors Global Excessive

Location	Value	Value (%)
no Source Counters data available. Kernel may not have been built with -lineinfo or directive not supported.		

Warp Stall Sampling (All Samples)

Location	Value	Value (%)
no Source Counters data available. Kernel may not have been built with -lineinfo or directive not supported.		

Most Instructions Executed

Location	Value	Value (%)
no Source Counters data available. Kernel may not have been built with -lineinfo or directive not supported.		

Follow the rules section to get guidance on how to navigate through the report and specific documentation references to kilobenchs in this kernel.
You could also disable Analysis Rules to focus on selected performance aspects and make profiling faster.